

PAULT DOCUMENTATION

2026 02 15 Polish Harden

Source: docs/plans/2026-02-15-polish-harden.md

Theme: Clean Technical

Generated: February 26, 2026

Polish & Harden Implementation Plan

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Fix critical bugs, add comprehensive test coverage, harden edge cases, and polish accessibility across the Pault macOS app.

Architecture: Pault is a SwiftUI + SwiftData macOS 14+ app. All mutations go through PromptService. Views use @Bindable, @Query, and @Environment(\.modelContext). Tests use Swift Testing framework with in-memory ModelContainer.

Tech Stack: Swift, SwiftUI, SwiftData, AppKit (NSTextView, NSPasteboard), Swift Testing, os.Logger

Phase 1: Critical Fixes

Task 1: Fix security-scoped resource leak in AttachmentManager

Files:

- Modify: Pault/AttachmentManager.swift:110-136
- Modify: Pault/AttachmentsStripView.swift:96-117, 143-161

Step 1: Replace resolveURL with closure-based withResolvedURL

In Pault/AttachmentManager.swift, replace the resolveURL method (lines 110-136) with:

```
// MARK: - Resolution

/// Execute an action with the resolved on-disk URL for an attachment.
/// For referenced (bookmarked) files, this correctly starts AND stops
/// security-scoped resource access via `defer`.
static func withResolvedURL(for attachment: Attachment, perform action: (URL) -> Void) {
    switch attachment.storageMode {
    case "embedded":
        guard let relativePath = attachment.relativePath else {
            logger.warning("withResolvedURL: No relativePath for embedded attachment '\(attachment.filename)')")
            return
        }
        let url = attachmentsBaseDirectory.appendingPathComponent(relativePath)
        action(url)

    case "referenced":
        guard let bookmarkData = attachment.bookmarkData else {
            logger.warning("withResolvedURL: No bookmarkData for referenced attachment '\(attachment.filename)')")
            return
        }
        var isStale = false
        do {
            let url = try URL(
                resolvingBookmarkData: bookmarkData,
                options: .withSecurityScope,
                relativeTo: nil,
                bookmarkDataIsStale: &isStale
            )
            action(url)
        } catch {
            logger.error("withResolvedURL: Failed to resolve bookmarkData for referenced attachment '\(attachment.filename)')")
        }
    }
}
```

```

)
if isStale {
    logger.warning("Stale bookmark for: \(attachment.filename)")
}
let didStart = url.startAccessingSecurityScopedResource()
defer { if didStart { url.stopAccessingSecurityScopedResource() } }
action(url)
} catch {
    logger.error("withResolvedURL: Failed to resolve bookmark for '\(attachment.filename)':
    \(error.localizedDescription)")
}

default:
    logger.warning("withResolvedURL: Unknown storageMode '\(attachment.storageMode)' for
    '\(attachment.filename)'")
}
}

/// Resolve the on-disk URL for an attachment (non-scoped, for embedded files only).
/// WARNING: For referenced files, prefer `withResolvedURL` to ensure proper resource
cleanup.
static func resolveURL(for attachment: Attachment) -> URL? {
    guard attachment.storageMode == "embedded",
    let relativePath = attachment.relativePath else { return nil }
    return attachmentsBaseDirectory.appendingPathComponent(relativePath)
}

```

Step 2: Update AttachmentsStripView call sites

In Pault/AttachmentsStripView.swift, update addFile(at:) method — replace lines 102-107:

```

// Generate thumbnail for images
if AttachmentManager.isImage(attachment.mediaType) {
    AttachmentManager.withResolvedURL(for: attachment) { resolvedURL in
        attachment.thumbnailData = AttachmentManager.generateThumbnail(for: resolvedURL)
    }
    if attachment.thumbnailData == nil {
        attachmentsLogger.warning("Failed to generate thumbnail for '\(attachment.filename)'")
    }
}
}
}

```

Update openAttachment (line 143-146):

```

private func openAttachment(_ attachment: Attachment) {
    AttachmentManager.withResolvedURL(for: attachment) { url in
        NSWorkspace.shared.open(url)
    }
}

```

Update quickLookAttachment (line 148-151):

```

private func quickLookAttachment(_ attachment: Attachment) {
    AttachmentManager.withResolvedURL(for: attachment) { url in
        NSWorkspace.shared.activateFileViewerSelecting([url])
    }
}

```

Update insertInline (line 153-161):

```
private func insertInline(_ attachment: Attachment) {
    AttachmentManager.withResolvedURL(for: attachment) { url in
        NotificationCenter.default.post(
            name: .insertInlineImage,
            object: nil,
            userInfo: ["url": url]
        )
    }
}
```

Step 3: Build and verify

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -destination 'platform=macOS' build 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 4: Commit

```
git add Pault/AttachmentManager.swift Pault/AttachmentsStripView.swift
git commit -m "fix: close security-scoped resource leak with closure-based
withResolvedURL API"
```

Task 2: Wire `.insertInlineImage` notification in `RichTextEditor`

Files:

- Modify: `Pault/RichTextEditor.swift:87-121`

Step 1: Add notification observer to Coordinator

In `Pault/RichTextEditor.swift`, add an `import os` at the top of the file (after `import AppKit`):

```
import os

private let richTextLogger = Logger(subsystem: "com.pault.app", category:
"RichTextEditor")
```

Replace the `Coordinator` class (lines 87-121) with:

```
final class Coordinator: NSObject, NSTextViewDelegate {
    var parent: RichTextEditor
    var isEditing = false
    weak var textView: NSTextView?
    private var imageObserver: NSObjectProtocol?

    init(parent: RichTextEditor) {
        self.parent = parent
        super.init()

        imageObserver = NotificationCenter.default.addObserver(
            forName: .insertInlineImage,
            object: nil,
            queue: .main
        ) { [weak self] notification in
            self?.handleInsertInlineImage(notification)
        }
    }

    deinit {
```

```
if let observer = imageObserver {
NotificationCenter.default.removeObserver(observer)
}
}

private func handleInsertInlineImage(_ notification: Notification) {
guard let textView,
let url = notification.userInfo?["url"] as? URL else {
richTextLogger.warning("insertInlineImage: Missing textView or URL")
return
}

guard let image = UIImage(contentsOf: url) else {
richTextLogger.error("insertInlineImage: Could not load image from
\(url.lastPathComponent)")
return
}

let attachment = NSTextAttachment()
let cell = NSTextAttachmentCell(imageCell: image)

// Scale image to fit within editor width
let maxWidth = textView.textContainer?.containerSize.width ?? 400
let scale = min(1.0, (maxWidth - 20) / image.size.width)
cell.cellSize = NSSize(
width: image.size.width * scale,
height: image.size.height * scale
)
attachment.attachmentCell = cell

let attrString = NSAttributedString(attachment: attachment)

let insertionPoint = textView.selectedRange().location
textView.textStorage?.insert(attrString, at: insertionPoint)

// Move cursor past the inserted image
textView.setSelectedRange(NSRange(location: insertionPoint + 1, length: 0))

syncContent()
richTextLogger.info("Inserted inline image: \(url.lastPathComponent)")
}

func textDidBeginEditing(_ notification: Notification) {
isEditing = true
}

func textDidEndEditing(_ notification: Notification) {
isEditing = false
syncContent()
}

func textDidChange(_ notification: Notification) {
syncContent()
}

private func syncContent() {
guard let textView else { return }
}
```

```
parent.plainContent = textView.string

let fullRange = NSRange(location: 0, length: textView.textStorage?.length ?? 0)
do {
    parent.attributedContent = try textView.textStorage?.data(
        from: fullRange,
        documentAttributes: [.documentType: NSAttributedString.DocumentType.rtf]
    )
} catch {
    richTextLogger.error("syncContent: Failed to serialize RTFD -
    \(error.localizedDescription)")
}
}
}
```

Step 2: Build and verify

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -destination 'platform=macOS' build 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Commit

```
git add Pault/RichTextEditor.swift
git commit -m "feat: wire insertInlineImage notification in RichTextEditor Coordinator"
```

Task 3: Fix force unwrap in AttachmentManager

Files:

- Modify: `Pault/AttachmentManager.swift:17-23`

Step 1: Replace `.first!` with `safe unwrap`

In `Pault/AttachmentManager.swift`, replace lines 17-23:

```
static var attachmentsBaseDirectory: URL {
    guard let appSupport = FileManager.default.urls(
        for: .applicationSupportDirectory,
        in: .userDomainMask
    ).first else {
        logger.fault("attachmentsBaseDirectory: Application Support directory not found")
        // Fallback to temp directory to avoid crash
        return FileManager.default.temporaryDirectory
            .appendingPathComponent("Pault/Attachments", isDirectory: true)
    }
    return appSupport.appendingPathComponent("Pault/Attachments", isDirectory: true)
}
```

Step 2: Build and verify

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -destination 'platform=macOS' build 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Run existing tests

```
Run: cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme
Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test
Suite|Tests|Passed|Failed)"
```

Expected: All tests pass

Step 4: Commit

```
git add Pault/AttachmentManager.swift
git commit -m "fix: replace force unwrap with safe fallback in attachmentsBaseDirectory"
```

Task 4: Silent error swallowing audit

Files:

- Modify: Pault/AppDelegate.swift:77-81
- Modify: Pault/AttachmentManager.swift:141-154
- Modify: Pault/PromptService.swift:59-77
- Modify: Pault/TemplateVariablesView.swift:115-133

Step 1: Fix AppDelegate.applicationWillTerminate

In Pault/AppDelegate.swift, add `import os` at line 2 (after `import SwiftUI`), and add a logger:

```
import os

private let appDelegateLogger = Logger(subsystem: "com.pault.app", category:
"AppDelegate")
```

Replace lines 77-81:

```
func applicationWillTerminate(_ notification: Notification) {
if let context = modelContainer?.mainContext {
do {
try context.save()
} catch {
appDelegateLogger.error("applicationWillTerminate: Failed to save –
\(error.localizedDescription)")
}
}
}
```

Step 2: Fix AttachmentManager.deleteFiles and deleteFile

In Pault/AttachmentManager.swift, replace `deleteFiles` (line ~141-144):

```
static func deleteFiles(for promptID: UUID) {
let dir = directory(for: promptID)
do {
try FileManager.default.removeItem(at: dir)
} catch {
logger.error("deleteFiles: Failed to remove directory for prompt \(promptID) –
\(error.localizedDescription)")
}
}
```

Replace `deleteFile` (line ~147-154):

```
static func deleteFile(for attachment: Attachment) {
guard attachment.storageMode == "embedded",
```

```

let relativePath = attachment.relativePath
else { return }

let url = attachmentsBaseDirectory.appendingPathComponent(relativePath)
do {
try FileManager.default.removeItem(at: url)
} catch {
logger.error("deleteFile: Failed to remove '\(attachment.filename)' –
\(\error.localizedDescription)")
}
}

```

Step 3: Fix PromptService.copyToClipboard

In Pault/PromptService.swift, replace the RTFD block in copyToClipboard (lines 68-74):

```

// If rich content exists, also provide RTFD
if let rtfdData = prompt.attributedContent {
do {
let attrString = try NSAttributedString(data: rtfdData, options: [.documentType:
NSAttributedString.DocumentType.rtfd], documentAttributes: nil)
let mutable = NSMutableAttributedString(attributedString: attrString)
let rtfdOutput = try mutable.data(from: NSRange(location: 0, length: mutable.length),
documentAttributes: [.documentType: NSAttributedString.DocumentType.rtfd])
pasteboard.setData(rtfdOutput, forType: .rtfd)
} catch {
serviceLogger.error("copyToClipboard: Failed to serialize RTFD –
\(\error.localizedDescription)")
}
}
}

```

Step 4: Fix TemplateVariablesView silent saves

In Pault/TemplateVariablesView.swift, add `import os` at line 3 (after `import SwiftData`):

```

import os

private let variablesLogger = Logger(subsystem: "com.pault.app", category:
"TemplateVariablesView")

```

Replace `debouncedSave` (lines 115-125):

```

private func debouncedSave() {
saveTask?.cancel()
saveTask = Task {
try? await Task.sleep(for: .milliseconds(500))
guard !Task.isCancelled else { return }
await MainActor.run {
prompt.updatedAt = Date()
do {
try modelContext.save()
} catch {
variablesLogger.error("debouncedSave: Failed – \(\error.localizedDescription)")
}
}
}
}
}
}

```

Replace `clearAllValues` (lines 127-133):

```
private func clearAllValues() {
    for variable in prompt.templateVariables {
        variable.defaultValue = ""
    }
    prompt.updatedAt = Date()
    do {
        try modelContext.save()
    } catch {
        variablesLogger.error("clearAllValues: Failed to save – \(error.localizedDescription)")
    }
    variablesLogger.info("Cleared all variable values for prompt '\(prompt.title)'")
}
```

Step 5: Build and verify

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -destination 'platform=macOS' build 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 6: Run all tests

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test Suite|Tests|Passed|Failed)"`

Expected: All tests pass

Step 7: Commit

```
git add Pault/AppDelegate.swift Pault/AttachmentManager.swift Pault/PromptService.swift
Pault/TemplateVariablesView.swift
git commit -m "fix: replace silent try? with do/catch + os.Logger across 4 files"
```

Phase 2: Test Coverage

Task 5: PromptService CRUD & mutation tests

Files:

- Create: `PaultTests/PromptServiceTests.swift`

Step 1: Write the test file

Create `PaultTests/PromptServiceTests.swift`:

```
//
// PromptServiceTests.swift
// PaultTests
//

import Testing
import SwiftData
import AppKit
@testable import Pault

struct PromptServiceTests {
```

```
private func makeContext() throws -> ModelContext {
    let container = try ModelContainer(
        for: Prompt.self, TemplateVariable.self, Tag.self, Attachment.self,
        configurations: ModelConfiguration(isStoredInMemoryOnly: true)
    )
    return ModelContext(container)
}

// MARK: - createPrompt

@Test func createPromptInsertsWithDefaults() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt()
    #expect(prompt.title == "")
    #expect(prompt.content == "")
    #expect(!prompt.isFavorite)
    #expect(!prompt.isArchived)
}

@Test func createPromptTrimsWhitespace() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: " Hello ", content: "\nWorld\n")
    #expect(prompt.title == "Hello")
    #expect(prompt.content == "World")
}

@Test func createPromptPersists() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    _ = service.createPrompt(title: "Test", content: "Body")

    let descriptor = FetchDescriptor<Prompt>()
    let results = try context.fetch(descriptor)
    #expect(results.count == 1)
    #expect(results.first?.title == "Test")
}

// MARK: - deletePrompt

@Test func deletePromptRemovesFromContext() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "To Delete", content: "")
    service.deletePrompt(prompt)

    let descriptor = FetchDescriptor<Prompt>()
    let results = try context.fetch(descriptor)
    #expect(results.isEmpty)
}

// MARK: - toggleFavorite
```

```
@Test func toggleFavoriteFlipsFlag() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Test", content: "")
    #expect(!prompt.isFavorite)

    service.toggleFavorite(prompt)
    #expect(prompt.isFavorite)

    service.toggleFavorite(prompt)
    #expect(!prompt.isFavorite)
}

@Test func toggleFavoriteUpdatesTimestamp() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Test", content: "")
    let before = prompt.updatedAt

    // Small delay to ensure timestamp difference
    Thread.sleep(forTimeInterval: 0.01)
    service.toggleFavorite(prompt)

    #expect(prompt.updatedAt > before)
}

// MARK: - toggleArchive

@Test func toggleArchiveFlipsFlag() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Test", content: "")
    #expect(!prompt.isArchived)

    service.toggleArchive(prompt)
    #expect(prompt.isArchived)

    service.toggleArchive(prompt)
    #expect(!prompt.isArchived)
}

// MARK: - Tag operations

@Test func addTagAppendsToPrompt() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Test", content: "")
    let tag = service.createTag(name: "Work")

    service.addTag(tag, to: prompt)
    #expect(prompt.tags.count == 1)
    #expect(prompt.tags.first?.name == "Work")
}
```

```
@Test func addTagPreventsDuplicates() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Test", content: "")
    let tag = service.createTag(name: "Work")

    service.addTag(tag, to: prompt)
    service.addTag(tag, to: prompt) // duplicate
    #expect(prompt.tags.count == 1)
}

@Test func removeTagRemovesFromPrompt() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Test", content: "")
    let tag = service.createTag(name: "Work")

    service.addTag(tag, to: prompt)
    service.removeTag(tag, from: prompt)
    #expect(prompt.tags.isEmpty)
}

// MARK: - createTag

@Test func createTagPersists() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let tag = service.createTag(name: "Personal", color: "red")
    #expect(tag.name == "Personal")
    #expect(tag.color == "red")

    let descriptor = FetchDescriptor<Tag>()
    let results = try context.fetch(descriptor)
    #expect(results.count == 1)
}

@Test func createTagDeduplicatesCaseInsensitive() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let tag1 = service.createTag(name: "Work")
    let tag2 = service.createTag(name: "work")
    let tag3 = service.createTag(name: "WORK")

    #expect(tag1.id == tag2.id)
    #expect(tag2.id == tag3.id)

    let descriptor = FetchDescriptor<Tag>()
    let results = try context.fetch(descriptor)
    #expect(results.count == 1)
}

@Test func createTagTrimsName() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)
```

```

let tag = service.createTag(name: " Spaced ")
#expect(tag.name == "Spaced")
}

// MARK: - copyToClipboard

@Test func copyToClipboardSetsPlainText() throws {
let context = try makeContext()
let service = PromptService(modelContext: context)

let prompt = service.createPrompt(title: "Test", content: "Hello World")
service.copyToClipboard(prompt)

let pasteboard = NSPasteboard.general
let text = pasteboard.string(forType: .string)
#expect(text == "Hello World")
}

@Test func copyToClipboardResolvesTemplateVariables() throws {
let context = try makeContext()
let service = PromptService(modelContext: context)

let prompt = service.createPrompt(title: "Test", content: "Hi {{name}}")
let variable = TemplateVariable(name: "name", defaultValue: "Alice")
context.insert(variable)
prompt.templateVariables.append(variable)

service.copyToClipboard(prompt)

let pasteboard = NSPasteboard.general
let text = pasteboard.string(forType: .string)
#expect(text == "Hi Alice")
}

@Test func copyToClipboardUpdatesLastUsedAt() throws {
let context = try makeContext()
let service = PromptService(modelContext: context)

let prompt = service.createPrompt(title: "Test", content: "Content")
#expect(prompt.lastUsedAt == nil)

service.copyToClipboard(prompt)
#expect(prompt.lastUsedAt != nil)
}
}

```

Step 2: Run tests

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test Suite|test|Passed|Failed)"`

Expected: All new and existing tests pass

Step 3: Commit

```
git add PaulTests/PromptServiceTests.swift
git commit -m "test: add PromptService CRUD & mutation tests (16 tests)"
```

Task 6: PromptService filter tests

Files:

- Modify: PaulTests/PromptServiceTests.swift

Step 1: Add filter tests to the existing file

Append to the end of PaulTests/PromptServiceTests.swift (inside the struct, before the closing }):

```
// MARK: - filterPrompts

@Test func filterExcludesArchivedByDefault() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let p1 = service.createPrompt(title: "Active", content: "")
    let p2 = service.createPrompt(title: "Archived", content: "")
    p2.isArchived = true
    try context.save()

    let result = service.filterPrompts([p1, p2])
    #expect(result.count == 1)
    #expect(result.first?.title == "Active")
}

@Test func filterShowsArchivedWhenRequested() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let p1 = service.createPrompt(title: "Active", content: "")
    let p2 = service.createPrompt(title: "Archived", content: "")
    p2.isArchived = true

    let result = service.filterPrompts([p1, p2], showArchived: true)
    #expect(result.count == 1)
    #expect(result.first?.title == "Archived")
}

@Test func filterFavoritesOnly() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let p1 = service.createPrompt(title: "Regular", content: "")
    let p2 = service.createPrompt(title: "Faved", content: "")
    p2.isFavorite = true

    let result = service.filterPrompts([p1, p2], showOnlyFavorites: true)
    #expect(result.count == 1)
    #expect(result.first?.title == "Faved")
}

@Test func filterRecentSortsAndCaps() throws {
```

```
let context = try makeContext()
let service = PromptService(modelContext: context)

var prompts: [Prompt] = []
for i in 0..  
5 {
    let p = service.createPrompt(title: "P\\(i)", content: "")
    p.lastUsedAt = Date().addingTimeInterval(Double(i) * 60)
    prompts.append(p)
}

let result = service.filterPrompts(prompts, showOnlyRecent: true, recentLimit: 3)
#expect(result.count == 3)
// Most recent first
#expect(result[0].title == "P4")
#expect(result[1].title == "P3")
#expect(result[2].title == "P2")
}

@Test func filterRecentExcludesNeverUsed() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let p1 = service.createPrompt(title: "Used", content: "")
    p1.lastUsedAt = Date()
    let p2 = service.createPrompt(title: "Never", content: "")

    let result = service.filterPrompts([p1, p2], showOnlyRecent: true)
    #expect(result.count == 1)
    #expect(result.first?.title == "Used")
}

@Test func filterByTag() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let tag = service.createTag(name: "Work")
    let p1 = service.createPrompt(title: "Tagged", content: "")
    let p2 = service.createPrompt(title: "Untagged", content: "")
    service.addTag(tag, to: p1)

    let result = service.filterPrompts([p1, p2], tagFilter: tag)
    #expect(result.count == 1)
    #expect(result.first?.title == "Tagged")
}

@Test func filterBySearchTextMatchesTitle() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let p1 = service.createPrompt(title: "Meeting Notes", content: "")
    let p2 = service.createPrompt(title: "Shopping List", content: "")

    let result = service.filterPrompts([p1, p2], searchText: "meeting")
    #expect(result.count == 1)
    #expect(result.first?.title == "Meeting Notes")
}

@Test func filterBySearchTextMatchesContent() throws {
```

```

let context = try makeContext()
let service = PromptService(modelContext: context)

let p1 = service.createPrompt(title: "A", content: "Hello world")
let p2 = service.createPrompt(title: "B", content: "Goodbye moon")

let result = service.filterPrompts([p1, p2], searchText: "hello")
#expect(result.count == 1)
#expect(result.first?.title == "A")
}

@Test func filterBySearchTextMatchesTagName() throws {
let context = try makeContext()
let service = PromptService(modelContext: context)

let tag = service.createTag(name: "Urgent")
let p1 = service.createPrompt(title: "A", content: "")
let p2 = service.createPrompt(title: "B", content: "")
service.addTag(tag, to: p1)

let result = service.filterPrompts([p1, p2], searchText: "urgent")
#expect(result.count == 1)
#expect(result.first?.title == "A")
}

@Test func filterMaxResultsCaps() throws {
let context = try makeContext()
let service = PromptService(modelContext: context)

var prompts: [Prompt] = []
for i in 0..<10 {
prompts.append(service.createPrompt(title: "P\(i)", content: ""))
}

let result = service.filterPrompts(prompts, maxResults: 3)
#expect(result.count == 3)
}

@Test func filterEmptyInputReturnsEmpty() throws {
let context = try makeContext()
let service = PromptService(modelContext: context)

let result = service.filterPrompts([])
#expect(result.isEmpty)
}

```

Step 2: Run tests

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test Suite|test|Passed|Failed)"`

Expected: All tests pass

Step 3: Commit

```

git add PaultTests/PromptServiceTests.swift
git commit -m "test: add PromptService filterPrompts tests (11 tests)"

```

Task 7: AttachmentManager file operation tests

Files:

- Create: PaulTests/AttachmentManagerFileTests.swift

Step 1: Write the test file

Create PaulTests/AttachmentManagerFileTests.swift:

```
//
// AttachmentManagerFileTests.swift
// PaulTests
//

import Testing
import Foundation
@testable import Pault

struct AttachmentManagerFileTests {

    private func createTempImage() throws -> URL {
        let tempDir = FileManager.default.temporaryDirectory
            .appendingPathComponent("PaulTests-\(UUID().uuidString)")
        try FileManager.default.createDirectory(at: tempDir, withIntermediateDirectories: true)

        let imageURL = tempDir.appendingPathComponent("test.png")

        // Create a minimal valid PNG (1x1 red pixel)
        let pngData = createMinimalPNG()
        try pngData.write(to: imageURL)

        return imageURL
    }

    private func createMinimalPNG() -> Data {
        // Create a 1x1 red pixel PNG via UIImage
        let image = UIImage(size: NSSize(width: 1, height: 1))
        image.lockFocus()
        NSColor.red.drawSwatch(in: NSRect(x: 0, y: 0, width: 1, height: 1))
        image.unlockFocus()

        guard let tiffData = image.tiffRepresentation,
            let rep = NSBitmapImageRep(data: tiffData),
            let pngData = rep.representation(using: .png, properties: [:])
        else {
            return Data()
        }
        return pngData
    }

    private func cleanup(_ url: URL) {
        try? FileManager.default.removeItem(at: url.deletingLastPathComponent())
    }

    // MARK: - storeFile (embedded)

    @Test func storeFileEmbeddedCopiesFile() throws {
```

```
let imageURL = try createTempImage()
defer { cleanup(imageURL) }

let promptID = UUID()
let attachment = try AttachmentManager.storeFile(at: imageURL, for: promptID)

#expect(attachment.storageMode == "embedded")
#expect(attachment.filename == "test.png")
#expect(attachment.fileSize > 0)
#expect(attachment.relativePath != nil)

// Verify file was actually copied
guard let relativePath = attachment.relativePath else {
    Issue.record("relativePath is nil")
    return
}
let copiedURL = AttachmentManager.attachmentsBaseDirectory
    .appendingPathComponent(relativePath)
#expect(FileManager.default.fileExists(atPath: copiedURL.path))

// Cleanup
AttachmentManager.deleteFiles(for: promptID)
}

// MARK: - resolveURL

@Test func resolveURLReturnsPathForEmbedded() throws {
    let imageURL = try createTempImage()
    defer { cleanup(imageURL) }

    let promptID = UUID()
    let attachment = try AttachmentManager.storeFile(at: imageURL, for: promptID)

    let resolved = AttachmentManager.resolveURL(for: attachment)
    #expect(resolved != nil)
    #expect(FileManager.default.fileExists(atPath: resolved!.path))

    AttachmentManager.deleteFiles(for: promptID)
}

@Test func resolveURLReturnsNilForReferencedAttachment() throws {
    // resolveURL is now embedded-only; referenced files should use withResolvedURL
    let attachment = Attachment(
        filename: "test.txt",
        mediaType: "public.plain-text",
        fileSize: 100,
        storageMode: "referenced",
        bookmarkData: nil
    )

    let resolved = AttachmentManager.resolveURL(for: attachment)
    #expect(resolved == nil)
}

// MARK: - deleteFiles

@Test func deleteFilesRemovesPromptDirectory() throws {
    let imageURL = try createTempImage()
```

```

defer { cleanup(imageURL) }

let promptID = UUID()
_ = try AttachmentManager.storeFile(at: imageURL, for: promptID)

let dir = AttachmentManager.directory(for: promptID)
#expect(FileManager.default.fileExists(atPath: dir.path))

AttachmentManager.deleteFiles(for: promptID)
#expect(!FileManager.default.fileExists(atPath: dir.path))
}

@Test func deleteFilesNoopForMissingDirectory() {
    // Should not throw/crash for a non-existent prompt directory
    let fakeID = UUID()
    AttachmentManager.deleteFiles(for: fakeID)
    // If we reach here without crashing, the test passes
}

// MARK: - generateThumbnail

@Test func generateThumbnailProducesData() throws {
    let imageURL = try createTempImage()
    defer { cleanup(imageURL) }

    let data = AttachmentManager.generateThumbnail(for: imageURL)
    #expect(data != nil)
    #expect((data?.count ?? 0) > 0)
}

@Test func generateThumbnailReturnsNilForNonImage() {
    let textURL = FileManager.default.temporaryDirectory
        .appendingPathComponent("test.txt")
    try? "hello".write(to: textURL, atomically: true, encoding: .utf8)
    defer { try? FileManager.default.removeItem(at: textURL) }

    let data = AttachmentManager.generateThumbnail(for: textURL)
    #expect(data == nil)
}

// MARK: - isImage

@Test func isImageRecognizesWebP() {
    #expect(AttachmentManager.isImage("org.webmproject.webp") == true)
}

@Test func isImageRejectsPDF() {
    #expect(AttachmentManager.isImage("com.adobe.pdf") == false)
}
}

```

Step 2: Run tests

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test Suite|test|Passed|Failed)"`

Expected: All tests pass

Step 3: Commit

```
git add PaulTests/AttachmentManagerFileTests.swift
git commit -m "test: add AttachmentManager file operation tests (9 tests)"
```

Task 8: Integration tests

Files:

- Create: PaulTests/IntegrationTests.swift

Step 1: Write the test file

Create PaulTests/IntegrationTests.swift:

```
//
// IntegrationTests.swift
// PaulTests
//

import Testing
import SwiftData
import AppKit
@testable import Pault

struct IntegrationTests {

    private func makeContext() throws -> ModelContext {
        let container = try ModelContainer(
            for: Prompt.self, TemplateVariable.self, Tag.self, Attachment.self,
            configurations: ModelConfiguration(isStoredInMemoryOnly: true)
        )
        return ModelContext(container)
    }

    // MARK: - Template Variables → Copy

    @Test func templateVariablesResolveOnCopy() throws {
        let context = try makeContext()
        let service = PromptService(modelContext: context)

        // Create prompt with template content
        let prompt = service.createPrompt(
            title: "Outreach",
            content: "Hi {{name}}, I work at {{company}}. Let's connect!"
        )

        // Sync variables (simulating what PromptDetailView does)
        TemplateEngine.syncVariables(for: prompt, in: context)
        #expect(prompt.templateVariables.count == 2)

        // Fill in values
        prompt.templateVariables.first(where: { $0.name == "name" })?.defaultValue = "Alice"
        prompt.templateVariables.first(where: { $0.name == "company" })?.defaultValue = "Acme"

        // Copy to clipboard
        service.copyToClipboard(prompt)

        // Verify clipboard has resolved text
```

```
let text = NSPasteboard.general.string(forType: .string)
#expect(text == "Hi Alice, I work at Acme. Let's connect!")
}

@Test func templateVariablesPartialFillLeavesMarkers() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(
        title: "Test",
        content: "{{greeting}} {{name}}"
    )
    TemplateEngine.syncVariables(for: prompt, in: context)

    // Only fill one variable
    prompt.templateVariables.first(where: { $0.name == "greeting" })?.defaultValue = "Hello"

    service.copyToClipboard(prompt)

    let text = NSPasteboard.general.string(forType: .string)
    #expect(text == "Hello {{name}}")
}

// MARK: - Cascade Deletes

@Test func deletePromptCascadesTemplateVariables() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Test", content: "{{var1}} {{var2}}")
    TemplateEngine.syncVariables(for: prompt, in: context)
    try context.save()

    #expect(prompt.templateVariables.count == 2)

    service.deletePrompt(prompt)

    let varDescriptor = FetchDescriptor<TemplateVariable>()
    let vars = try context.fetch(varDescriptor)
    #expect(vars.isEmpty)
}

// MARK: - Tag Filtering

@Test func tagFilterFindTaggedPrompt() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let tag = service.createTag(name: "Urgent")
    let p1 = service.createPrompt(title: "Tagged", content: "")
    let p2 = service.createPrompt(title: "Untagged", content: "")
    service.addTag(tag, to: p1)

    let result = service.filterPrompts([p1, p2], tagFilter: tag)
    #expect(result.count == 1)
    #expect(result.first?.title == "Tagged")
}
```

```
// MARK: - Variable Sync Lifecycle

@Test func variableSyncAddAndRemove() throws {
    let context = try makeContext()

    let prompt = Prompt(title: "Test", content: "{{name}} from {{company}}")
    context.insert(prompt)

    // Initial sync
    TemplateEngine.syncVariables(for: prompt, in: context)
    #expect(prompt.templateVariables.count == 2)

    // User fills in name
    prompt.templateVariables.first(where: { $0.name == "name" })?.defaultValue = "Bob"

    // User removes {{company}} from content
    prompt.content = "Hello {{name}}!"
    TemplateEngine.syncVariables(for: prompt, in: context)

    #expect(prompt.templateVariables.count == 1)
    #expect(prompt.templateVariables.first?.name == "name")
    #expect(prompt.templateVariables.first?.defaultValue == "Bob") // value preserved

    // User adds a new variable
    prompt.content = "Hello {{name}} at {{role}}!"
    TemplateEngine.syncVariables(for: prompt, in: context)

    #expect(prompt.templateVariables.count == 2)
    let roleVar = prompt.templateVariables.first(where: { $0.name == "role" })
    #expect(roleVar != nil)
    #expect(roleVar?.defaultValue == "") // new variable starts empty
}
}
```

Step 2: Run all tests

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test Suite|test|Passed|Failed)"`

Expected: All tests pass

Step 3: Commit

```
git add PaultTests/IntegrationTests.swift
git commit -m "test: add integration tests for template lifecycle and cascades (5 tests)"
```

Phase 3: Edge Case Hardening

Task 9: ExpandingTextEditor height coalescing

Files:

- Modify: `Pault/ExpandingTextEditor.swift:62-67,69-87`

Step 1: Add coalescing flag and debounced recalculation

In `Pault/ExpandingTextEditor.swift`, add a property to the `Coordinator` class after the existing properties (after line 53):

```
private var heightRecalcPending = false
```

Replace `textDidChange` (lines 62-67):

```
func textDidChange(_ notification: Notification) {
    guard let textView else { return }
    text.wrappedValue = textView.string
    updatePlaceholder()
    scheduleHeightRecalc()
}

private func scheduleHeightRecalc() {
    guard !heightRecalcPending else { return }
    heightRecalcPending = true
    DispatchQueue.main.async { [weak self] in
        self?.heightRecalcPending = false
        self?.recalculateHeight()
    }
}
```

Step 2: Build and verify

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -destination 'platform=macOS' build 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Run tests

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Passed|Failed)"`

Expected: All tests pass

Step 4: Commit

```
git add Pault/ExpandingTextEditor.swift
git commit -m "perf: coalesce ExpandingTextEditor height recalculation to prevent layout thrashing"
```

Task 10: Atomic file write in `AttachmentManager.storeEmbedded`

Files:

- Modify: `Pault/AttachmentManager.swift:56-81`

Step 1: Replace direct copy with atomic write-then-replace

In `Pault/AttachmentManager.swift`, replace `storeEmbedded` (lines 56-81):

```
private static func storeEmbedded(
    sourceURL: URL,
    promptID: UUID,
    filename: String,
    mediaType: String,
    fileSize: Int64
) throws -> Attachment {
```

```

let destDir = directory(for: promptID)
try FileManager.default.createDirectory(at: destDir, withIntermediateDirectories: true)

let attachmentID = UUID()
let ext = sourceURL.pathExtension
let destURL = destDir.appendingPathComponent("\(attachmentID.uuidString).\(ext)")

// Atomic write: copy to temp file, then replace
let tempURL = destDir.appendingPathComponent("\(attachmentID.uuidString).\(ext).tmp")
try FileManager.default.copyItem(at: sourceURL, to: tempURL)

if FileManager.default.fileExists(atPath: destURL.path) {
    _ = try FileManager.default.replaceItemAt(destURL, withItemAt: tempURL)
} else {
    try FileManager.default.moveItem(at: tempURL, to: destURL)
}

logger.info("Embedded file: \(filename) (\(fileSize) bytes)")

let relativePath = "\(promptID.uuidString)/\(attachmentID.uuidString).\(ext)"
return Attachment(
    id: attachmentID,
    filename: filename,
    mediaType: mediaType,
    fileSize: fileSize,
    storageMode: "embedded",
    relativePath: relativePath
)
}

```

Step 2: Build and run tests

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Passed|Failed)"`

Expected: All tests pass

Step 3: Commit

```

git add Pault/AttachmentManager.swift
git commit -m "fix: use atomic write-then-move for embedded attachment storage"

```

Task 11: Broader drag-drop types in RichTextEditor

Files:

- Modify: `Pault/RichTextEditor.swift:37`

Step 1: Register additional pasteboard types

In `Pault/RichTextEditor.swift`, replace line 37:

```
textView.registerForDraggedTypes([.fileURL, .URL, .png, .tiff])
```

Step 2: Add logging to RichEditorTextView drag handler

Replace `performDragOperation` in `RichEditorTextView` (lines 130-145):

```

override func performDragOperation(_ sender: any NSDraggingInfo) -> Bool {
    if let onImageDrop {
        let pasteboard = sender.draggingPasteboard

        // Handle file URLs
        if let urls = pasteboard.readObjects(forClasses: [NSURL.self]) as? [URL] {
            for url in urls {
                if isImageURL(url) {
                    onImageDrop(url)
                } else {
                    richTextLogger.info("Ignored non-image drop: \(url.lastPathComponent)")
                }
            }
        }
    }

    return super.performDragOperation(sender)
}

```

Also update `isImageURL` to add `webp`:

```

private func isImageURL(_ url: URL) -> Bool {
    let imageExtensions: Set<String> = ["png", "jpg", "jpeg", "tiff", "tif", "gif", "bmp",
    "webp", "heic", "heif"]
    return imageExtensions.contains(url.pathExtension.lowercased())
}

```

Step 3: Build and verify

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -destination 'platform=macOS' build 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 4: Commit

```

git add Pault/RichTextEditor.swift
git commit -m "feat: expand drag-drop to support .URL and more image formats"

```

Task 12: Tab key navigation in `TemplateVariablesView`

Files:

- Modify: `Pault/TemplateVariablesView.swift`

Step 1: Add `FocusState` and apply to variable fields

In `Pault/TemplateVariablesView.swift`, add after the `@Bindable var prompt` line (line 14):

```
@FocusState private var focusedVariableID: UUID?
```

Update the `ExpandingTextEditor` inside the Grid's `ForEach` (replace lines 69-83):

```

ExpandingTextEditor(
    text: Binding(
        get: { variable.defaultValue },
        set: { newValue in
            variable.defaultValue = newValue
            debouncedSave()
        }
    )
)

```

```

),
placeholder: "Enter \{(variable.name)...\}"
)
.focused($focusedVariableID, equals: variable.id)
.frame(minHeight: 30)
.overlay(
  RoundedRectangle(cornerRadius: 6)
  .stroke(
    focusedVariableID == variable.id
    ? Color.accentColor
    : Color.secondary.opacity(0.3),
    lineWidth: 1
  )
)
)

```

Step 2: Build and verify

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -destination 'platform=macOS' build 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Commit

```

git add Pault/TemplateVariablesView.swift
git commit -m "feat: add FocusState for Tab key navigation in TemplateVariablesView"

```

Task 13: Guard against whitespace-only variable names

Files:

- Modify: `Pault/TemplateEngine.swift:18-28`
- Modify: `PaultTests/TemplateEngineTests.swift`

Step 1: Write the failing test

Add to `PaultTests/TemplateEngineTests.swift` (inside the struct, after the existing extract tests):

```

@Test func extractIgnoresWhitespaceOnlyNames() {
  let names = TemplateEngine.extractVariableNames(from: "Hello {{ }} and {{ }} world")
  #expect(names.isEmpty)
}

```

Step 2: Run test to verify it fails

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "extractIgnoresWhitespace"`

Expected: The test may pass or fail depending on regex `\w+` behavior — `\w+` already requires at least one word character, so `{{ }}` with spaces won't match. But `{{ name }}` (spaces around word) needs testing too. Let's check and add a more complete guard.

Step 3: Update the regex pattern to also handle `{{ name }}` (spaces around variable)

In `Pault/TemplateEngine.swift`, replace the pattern (line 15) and the `extractVariableNames` method:

```

private static let variablePattern = /\{\{s*(\w+)\s*\}\}/

```

This allows `{{ name }}` to match as "name" (trimming whitespace inside braces).

Add another test:

```
@Test func extractTrimsSpacesInsideBraces() {
    let names = TemplateEngine.extractVariableNames(from: "{{ name }} and {{ company }}")
    #expect(names == ["name", "company"])
}
```

Also update the `resolve` method's pattern to match the same:

```
private static let variablePattern = /\{\{\s*(\w+)\s*\}\}/
```

Since both methods use the same static pattern, this single change fixes both.

Step 4: Run all tests

```
Run: cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme
Pault -destination 'platform=macOS' 2>&1 | grep -E "(Passed|Failed)"
```

Expected: All tests pass

Step 5: Commit

```
git add Pault/TemplateEngine.swift PaultTests/TemplateEngineTests.swift
git commit -m "fix: handle whitespace inside {{variable}} braces, reject empty names"
```

Phase 4: UI Polish

Task 14: Accessibility labels

Files:

- Modify: `Pault/AttachmentsStripView.swift`
- Modify: `Pault/TemplateVariablesView.swift`
- Modify: `Pault/PromptDetailView.swift`
- Modify: `Pault/InspectorView.swift`

Step 1: AttachmentsStripView accessibility

In `AttachmentThumbnailView`, add after the closing `}` of the `VStack` (after line 192):

```
.accessibilityLabel("\(attachment.filename)")
.accessibilityHint("Right-click for options")
```

On the "Add attachment" button (after `.help("Add attachment")` on line 42):

```
.accessibilityLabel("Add attachment")
```

Step 2: TemplateVariablesView accessibility

On the variable label `Text(variable.name)` (line 63), add:

```
.accessibilityHidden(true) // Label is read via the text field's accessibilityLabel
```

On the `ExpandingTextEditor` for each variable, add (after `.overlay(...)`):

```
.accessibilityLabel("Value for \(variable.name)")
```

On the "Clear" button (line 50), add:

```
.accessibilityLabel("Clear all variable values")
```

Step 3: PromptDetailView accessibility

On the inspector toggle button (after `.help("Toggle Inspector (Cmd+I)")` on line 82):

```
.accessibilityLabel(showInspector ? "Hide Inspector" : "Show Inspector")
```

Step 4: InspectorView accessibility

On the favorite toggle button (line 66-69), add:

```
.accessibilityLabel(prompt.isFavorite ? "Remove from favorites" : "Add to favorites")
```

On the archive button (line 109-111), add:

```
.accessibilityLabel(prompt.isArchived ? "Unarchive prompt" : "Archive prompt")
```

Step 5: Build and verify

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -destination 'platform=macOS' build 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 6: Commit

```
git add Pault/AttachmentsStripView.swift Pault/TemplateVariablesView.swift
Pault/PromptDetailView.swift Pault/InspectorView.swift
git commit -m "a11y: add accessibility labels to interactive elements across 4 views"
```

Task 15: TemplateVariablesView diagnostic logging

This was already addressed in Task 4 (silent error swallowing audit). The `variablesLogger` and `do/catch` patterns were added there. Mark as complete — no additional work needed.

Task 16: Consistent empty states

Files:

- Modify: `Pault/AttachmentsStripView.swift`

Step 1: Add empty state hint to attachment strip

In `Pault/AttachmentsStripView.swift`, replace the `if !sortedAttachments.isEmpty` block (lines 45-62) with:

```
if sortedAttachments.isEmpty {
  HStack {
    Spacer()
    VStack(spacing: 4) {
      Image(systemName: "arrow.down.doc")
        .font(.title3)
        .foregroundColor(.tertiary)
      Text("Drop files here or click +")
        .font(.caption)
        .foregroundColor(.tertiary)
    }
    .padding(.vertical, 12)
    Spacer()
  }
```

```

} else {
  ScrollView(.horizontal, showsIndicators: false) {
    HStack(spacing: 8) {
      ForEach(sortedAttachments) { attachment in
        AttachmentThumbnailView(attachment: attachment)
          .contextMenu {
            Button("Open") { openAttachment(attachment) }
            Button("Quick Look") { quickLookAttachment(attachment) }
            if AttachmentManager.isImage(attachment.mediaType) {
              Button("Insert Inline") { insertInline(attachment) }
            }
          }
        Divider()
        Button("Delete", role: .destructive) { deleteAttachment(attachment) }
      }
    }
  }
}
}
}
}

```

Step 2: Build and verify

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -destination 'platform=macOS' build 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Commit

```

git add Pault/AttachmentsStripView.swift
git commit -m "ui: add empty state hint to attachments strip"

```

Task 17: Final verification

Step 1: Run full test suite

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test Suite|Executed|Passed|Failed)"`

Expected: All tests pass. Target: ≥ 40 total tests (existing ~ 21 + ~ 41 new)

Step 2: Build Release config

Run: `cd /Users/dev/Documents/Software/macOS/Pault && xcodebuild -scheme Pault -configuration Release -destination 'platform=macOS' build 2>&1 | tail -10`

Expected: BUILD SUCCEEDED with zero warnings

Step 3: Review all changes

Run: `cd /Users/dev/Documents/Software/macOS/Pault && git log --oneline -20`

Verify all commits are present and well-described.

Summary

Phase	Tasks	New Tests	Files Modified
-----	-----	-----	-----
1: Critical Fixes	1–4	0	5 files
2: Test Coverage	5–8	~41	3 new test files
3: Edge Cases	9–13	2	4 files
4: UI Polish	14–17	0	5 files
Total	17	~43	~13 files